

- [License Code System — Frontend Integration Guide](#)
 - [Table of Contents](#)
 - [Overview](#)
 - [What Changed](#)
 - [Key Concepts](#)
 - [Roles](#)
 - [authorizedEmail](#)
 - [API Endpoints](#)
 - [1. Initialize Payment \(with Student Emails\)](#)
 - [Request Body](#)
 - [Field Reference](#)
 - [Validation Rules](#)
 - [Response \(Success\)](#)
 - [2. Verify Payment](#)
 - [Request Body](#)
 - [What happens on the backend](#)
 - [Response](#)
 - [3. Redeem License Code](#)
 - [Request Body](#)
 - [Authorization Logic](#)
 - [Success Response](#)
 - [Error Responses](#)
 - [4. Get My Institutional Codes](#)
 - [Response](#)
 - [Key Fields for the Frontend](#)
 - [5. Get Institutional Code Redemptions](#)
 - [Response](#)
 - [Error Responses](#)
 - [Data Flow](#)
 - [Buyer Journey \(Purchasing for Students\)](#)
 - [Student Journey \(Redeeming a Code\)](#)
 - [Error Handling](#)
 - [Error Response Format](#)
 - [Common Error Scenarios & Frontend UX](#)
 - [Frontend Implementation Notes](#)
 - [1. Buyer Purchase Form \(BODY Subscription\)](#)
 - [2. Buyer Dashboard — Code Management](#)

- [3. Student Redemption Form](#)
- [4. Handling the 403 in Code](#)
- [5. Optional: Validate Code Before Redeeming](#)
- [Quick Reference](#)

License Code System — Frontend Integration Guide

Last updated: 2026-02-23 **Scope:** Email-based license codes for institutional (BODY) subscriptions

Table of Contents

- [Overview](#)
 - [Key Concepts](#)
 - [API Endpoints](#)
 - [1. Initialize Payment \(with Student Emails\)](#)
 - [2. Verify Payment](#)
 - [3. Redeem License Code](#)
 - [4. Get My Institutional Codes](#)
 - [5. Get Institutional Code Redemptions](#)
 - [Data Flow](#)
 - [Error Handling](#)
 - [Frontend Implementation Notes](#)
-

Overview

The license code system allows a **buyer** (institution, school, organization) to purchase a BODY subscription for multiple students. The buyer provides a list of student emails during checkout, and the backend generates **one unique license code per email**. Each code can **only be redeemed by the student whose email matches the `authorizedEmail`** on that code.

What Changed

Before	After
Single license code per BODY purchase	One code per student email
Any student could redeem a code	Only the authorized email can redeem
Buyer saw one code with N redemptions	Buyer sees N codes mapped to N emails
Code format: INST-XXXX-XXXX	Code format: LIC-XXXX-XXXX (per-email)

Backward compatible: If no **studentEmails** are provided, the old single bulk code (**INST-XXXX-XXXX**) is still generated.

Key Concepts

Roles

Role	Description
Buyer	The student/institution purchasing the BODY subscription. They provide student emails and see the code-to-email mapping.
Student	A student who receives a license code (via the buyer or email). They redeem the code to activate their subscription.

authorizedEmail

A new field on each **LicenseCode** record:

- **Type:** **String | null**
- **Purpose:** Restricts redemption to the specified email address
- **Behavior:**
 - If set → only the student whose account email matches can redeem
 - If **null** → any student can redeem (old behavior, bulk codes)
- **Case-insensitive:** **Student@Example.com** matches **student@example.com**

API Endpoints

1. Initialize Payment (with Student Emails)

Creates a payment transaction for a BODY subscription with per-student email assignment.

POST /payment/initialize

Request Body

```
{
  "studentId": "buyer-student-uuid",
  "subscriptionId": "plan-uuid",
  "amount": 15000,
  "subscriptionType": "BODY",
  "numberOfSubjects": 4,
  "numberOfStudents": 3,
  "studentEmails": [
    "student1@example.com",
    "student2@example.com",
    "student3@example.com"
  ],
  "schoolType": "SECONDARY_SCHOOL",
  "examType": "WAEC",
  "numberOfDays": 30,
  "metadata": {
    "callbackUrl": "https://your-app.com/payment/verify"
  }
}
```

Field Reference

Field	Type	Required	Description
studentId	UUID		ID of the buyer (the person paying)
subscriptionId	UUID		ID of the subscription plan
amount	number		Payment amount in kobo

Field	Type	Required	Description
<code>subscriptionType</code>	<code>enum</code>	✓	Must be "BODY" for institutional
<code>numberOfSubjects</code>	<code>int</code>	✓	Number of subjects per student
<code>numberOfStudents</code>	<code>int</code>	✓ for BODY	Total number of students
<code>studentEmails</code>	<code>string[]</code>	✗ (optional)	Array of valid email addresses
<code>schoolType</code>	<code>string</code>	✗	e.g. "SECONDARY_SCHOOL"
<code>examType</code>	<code>string</code>	✗	e.g. "WAEC"
<code>numberOfDays</code>	<code>int</code>	✗	Custom duration in days
<code>promoCode</code>	<code>string</code>	✗	Promotional code
<code>useReferralBenefit</code>	<code>boolean</code>	✗	Apply referral discount
<code>metadata</code>	<code>object</code>	✗	Additional metadata (e.g. <code>callbackUrl</code>)

Validation Rules

Rule	Error
<code>subscriptionType</code> is BODY but <code>numberOfStudents</code> is missing	400 Bad Request: "Number of students is required for BODY subscriptions"
<code>studentEmails.length</code> != <code>numberOfStudents</code>	400 Bad Request: "Number of student emails (X) must match number of students (Y)"
<code>studentEmails</code> contains duplicates	400 Bad Request: "Duplicate student emails are not allowed"
Any email in <code>studentEmails</code> is not a valid email	400 Bad Request: (class-validator error)

Response (Success)

```
{
  "paymentUrl": "https://checkout.paystack.com/xxxxxx",

```

```
"reference": "paystack-ref-xxx",  
"accessCode": "abc123",  
"transactionId": "transaction-uuid"  
}
```

Frontend action: Redirect the user to `paymentUrl` to complete payment.

2. Verify Payment

After the buyer completes payment, verify it to trigger license code generation.

```
POST /payment/verify
```

Request Body

```
{  
  "reference": "paystack-ref-xxx"  
}
```

What happens on the backend

When verification succeeds for a BODY subscription:

If **studentEmails** were provided:

- Generates one `LicenseCode` per email
- Each code has `authorizedEmail` set to the corresponding email
- Each code has `maxRedemptions: 1`
- Code format: `LIC-XXXX-XXXX`
- An email is sent to the **buyer** with the full code→email mapping

If no **studentEmails** were provided (fallback):

- Generates one single `LicenseCode` with `maxRedemptions: numberOfStudents`
- `authorizedEmail` is `null` (any student can redeem)
- Code format: `INST-XXXX-XXXX`

Response

```
{
  "verified": true,
  "transaction": {
    "id": "transaction-uuid",
    "reference": "paystack-ref-xxx",
    "status": "SUCCESS",
    "amount": 15000
  }
}
```

3. Redeem License Code

A student submits a license code to activate their subscription.

```
POST /payment/redeem-license
```

Request Body

```
{
  "code": "LIC-ABCD-1234",
  "studentId": "student-uuid",
  "subjects": ["subj-uuid-1", "subj-uuid-2"],
  "courses": []
}
```

Field	Type	Required	Description
code	string	✓	The license code to redeem
studentId	UUID	✓	ID of the student redeeming
subjects	string[]	✗	Subject UUIDs to enroll in
courses	string[]	✗	Course UUIDs to enroll in

Authorization Logic

The backend performs these checks **in order**:

1. **Code exists** → 404 Not Found if not

2. **Code is active** → **400 Bad Request** if inactive
3. **Code has not expired** → **400 Bad Request** if expired
4. **Max redemptions not reached** → **400 Bad Request** if fully redeemed
5. **Student exists** → **404 Not Found** if not found
6. ★ **Email matches `authorizedEmail`** → **403 Forbidden** if mismatch

The email check is **case-insensitive**. If `authorizedEmail` is `null`, any student can redeem.

Success Response

Returns the created `StudentSubscription`:

```
{
  "id": "student-subscription-uuid",
  "studentId": "student-uuid",
  "subscriptionId": "sub-uuid",
  "startDate": "2026-02-23T00:00:00.000Z",
  "endDate": "2026-03-25T00:00:00.000Z",
  "status": "ACTIVE",
  "subjects": ["subj-uuid-1", "subj-uuid-2"],
  "courses": [],
  "examType": "WAEC",
  "paymentMethod": "LICENSE_CODE",
  "licenseCodeId": "license-code-uuid"
}
```

Error Responses

Status	Condition	Message
404	Code does not exist	"Invalid license code"
400	Code is inactive	"License code is inactive"
400	Code is expired	"License code has expired"
400	Max redemptions reached	"License code has reached maximum redemptions"
404	Student not found	"Student not found"
403	Email mismatch	"This license code is not authorized for your email address"

Status	Condition	Message
400	Too many subjects selected	"You can only select up to X subjects/courses for this license"

4. Get My Institutional Codes

Returns all license codes purchased by the currently authenticated student (the buyer).

```
GET /payment/institutional-codes
```

Auth: Requires **Bearer** `{{token}}` (student JWT)

Response

Returns an array of license codes with the **code**→**email mapping**:

```
[
  {
    "id": "license-code-uuid-1",
    "code": "LIC-ABCD-1234",
    "authorizedEmail": "student1@example.com",
    "isActive": true,
    "redemptionCount": 0,
    "maxRedemptions": 1,
    "redeemedBy": null,
    "redeemedAt": null,
    "expiresAt": null,
    "createdAt": "2026-02-23T12:00:00.000Z",
    "subscription": {
      "name": "WAEK Pro Plan",
      "duration": 365,
      "basePrice": 5000
    },
    "_count": {
      "redemptions": 0
    }
  },
  {
    "id": "license-code-uuid-2",
    "code": "LIC-EFGH-5678",
    "authorizedEmail": "student2@example.com",
    "isActive": true,
    "redemptionCount": 1,
    "maxRedemptions": 1,
    "redeemedBy": "redeemer-student-uuid",
```

```
"redeemedAt": "2026-02-23T14:30:00.000Z",
"expiresAt": null,
"createdAt": "2026-02-23T12:00:00.000Z",
"subscription": {
  "name": "WAEK Pro Plan",
  "duration": 365,
  "basePrice": 5000
},
"_count": {
  "redemptions": 1
}
}
```

Key Fields for the Frontend

Field	Description
<code>code</code>	The license code string to share with the student
<code>authorizedEmail</code>	Which email can redeem this code (<code>null</code> = anyone)
<code>isActive</code>	Whether the code can still be redeemed
<code>redemptionCount</code> / <code>maxRedemptions</code>	Progress: redeemed vs allowed
<code>redeemedBy</code>	Student ID of who redeemed (if redeemed)
<code>redeemedAt</code>	When it was redeemed (if redeemed)
<code>_count.redemptions</code>	Number of actual subscription records created

5. Get Institutional Code Redemptions

Returns detailed redemption history for a specific license code. Only accessible by the buyer who purchased it.

```
GET /payment/institutional-codes/:id/redemptions
```

Auth: Requires `Bearer {{token}}` (student JWT)

Param	Type	Description
id	UUID	The license code ID (not the code string)

Response

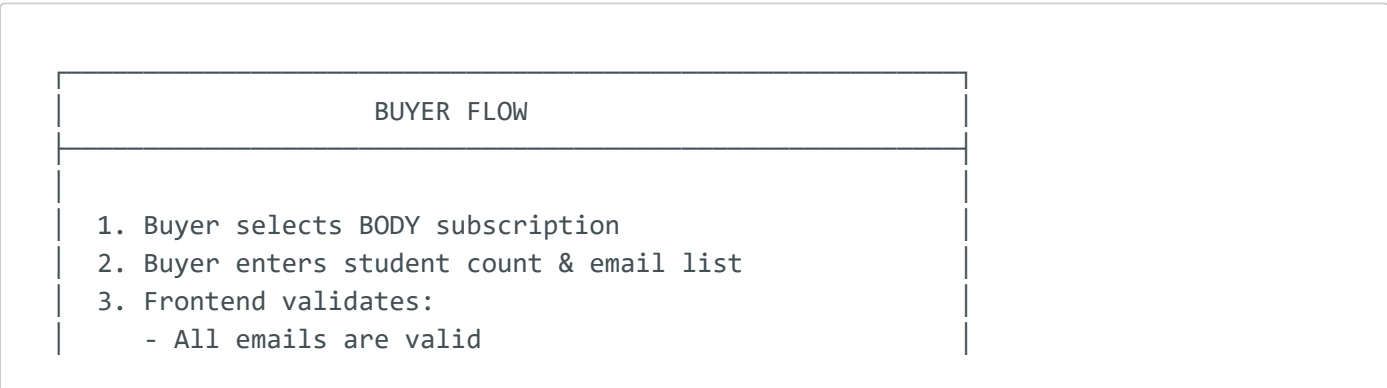
```
[
  {
    "id": "student-subscription-uuid",
    "studentId": "redeemer-student-uuid",
    "subscriptionId": "sub-uuid",
    "startDate": "2026-02-23T14:30:00.000Z",
    "endDate": "2027-02-23T14:30:00.000Z",
    "status": "ACTIVE",
    "student": {
      "id": "redeemer-student-uuid",
      "fullName": "Jane Doe",
      "email": "student2@example.com",
      "registrationDate": "2026-01-15T00:00:00.000Z"
    }
  }
]
```

Error Responses

Status	Condition	Message
404	License code not found	"License code not found"
400	Caller is not the purchaser	"You do not own this license code"

Data Flow

Buyer Journey (Purchasing for Students)



- No duplicates
 - Count matches numberOfStudents
4. POST /payment/initialize (with studentEmails)
 5. Buyer redirected to Paystack checkout
 6. After payment → POST /payment/verify
 7. Backend generates LIC-XXXX-XXXX per email
 8. Buyer receives email with code→email mapping
 9. Buyer views codes via GET /institutional-codes
 10. Buyer shares each code with the assigned student

Student Journey (Redeeming a Code)

STUDENT FLOW

1. Student receives a license code from buyer
2. Student enters the code in the redemption form
3. POST /payment/redeem-license
4. Backend checks:
 - ✓ Code is valid, active, not expired
 - ✓ Student's email ↔ code's authorizedEmail
5. If match → subscription activated
6. If mismatch → 403 "Not authorized for your email"

Error Handling

Error Response Format

All errors follow the standard NestJS exception format:

```
{
  "statusCode": 403,
  "message": "This license code is not authorized for your email address",
  "error": "Forbidden"
}
```

Common Error Scenarios & Frontend UX

Scenario	HTTP Status	User-Facing Message
Invalid code	404	"This license code is invalid. Please check and try again."
Code inactive	400	"This license code is no longer active."
Code expired	400	"This license code has expired."
Fully redeemed	400	"This license code has already been used."
Wrong email	403	"This code was assigned to a different email address. Please contact your institution."
Too many subjects	400	"You can only select up to X subjects with this code."
Duplicate emails in purchase	400	"Each student must have a unique email address."
Email count mismatch	400	"The number of emails must match the number of students."

Frontend Implementation Notes

1. Buyer Purchase Form (BODY Subscription)

When `subscriptionType === "BODY"`:

Institutional Purchase

Number of Students: [3]

Student Emails:

student1@school.edu[x]

student2@school.edu[x]

student3@school.edu[x]

[+ Add Another Email]

⚠ Each student will receive a unique code that only they can redeem.

[Proceed to Payment →]

Frontend validations before submitting:

- Each email is a valid email format
- No duplicate emails (case-insensitive comparison)
- Number of emails === `numberOfStudents`
- All emails are trimmed and lowercased

2. Buyer Dashboard — Code Management

After purchase, show the code→email mapping table:

Your Institutional Codes			
Code	Assigned To	Status	Redeemed
LIC-ABCD-1234	student1@school.edu	 Pending	—
LIC-EFGH-5678	student2@school.edu	 Redeemed	Feb 23
LIC-IJKL-9012	student3@school.edu	 Pending	—

Status logic:

```
function getCodeStatus(code) {  
  if (code.redemptionCount >= code.maxRedemptions) return "Redeemed";  
  if (!code.isActive) return "Inactive";  
  if (code.expiresAt && new Date(code.expiresAt) < new Date()) return "Expired";  
  return "Pending";  
}
```

Data source: `GET /payment/institutional-codes`

3. Student Redemption Form

Redeem License Code

Enter your license code:

LIC-ABCD-1234

Select your subjects: (optional)

☒ Mathematics

☒ English Language

☐ Physics

☐ Chemistry

[Redeem Code →]

On **403 error** (email mismatch), show:

 This license code was assigned to a specific email address that doesn't match your account. Please contact your institution or use the code assigned to your email.

4. Handling the 403 in Code

```
async function redeemLicenseCode(code: string, studentId: string, subjects:
string[]) {
  try {
    const response = await api.post("/payment/redeem-license", {
      code,
      studentId,
      subjects,
      courses: [],
    });
    // Success → navigate to dashboard
    return response.data;
  } catch (error) {
    if (error.response?.status === 403) {
      // Email mismatch – code is assigned to someone else
      showError("This code was assigned to a different email. Contact your
institution.");
    } else if (error.response?.status === 404) {
      showError("Invalid license code. Please check and try again.");
    } else if (error.response?.status === 400) {
      // Could be: inactive, expired, fully redeemed, or subject limit
      showError(error.response.data.message);
    }
  }
}
```

```
}  
}
```

5. Optional: Validate Code Before Redeeming

The buyer dashboard or student can visually preview a code's details. The backend doesn't have a dedicated public validation endpoint for students, but you can use the **403** response from `redeem-license` to detect email mismatches. The admin API does have `POST /admin/license-codes/validate`.

Quick Reference

Action	Method	Endpoint	Auth
Purchase (with emails)	POST	<code>/payment/initialize</code>	None
Verify payment	POST	<code>/payment/verify</code>	None
Redeem code	POST	<code>/payment/redeem-license</code>	None
View my codes (buyer)	GET	<code>/payment/institutional-codes</code>	JWT
View redemption history	GET	<code>/payment/institutional-codes/:id/redemptions</code>	JWT